

Smart Traffic Prediction and Route Optimization

By Anushka Ojha, Mehul Agarwal, Mayank Chopra

GitHub: <https://github.com/chopra-mayank/Traffic-Prediction>

Deployed Link : <https://traffic-frontend.kindsy-c26eed2e.southeastasia.azurecontainerapps.io/>

1. Project Overview

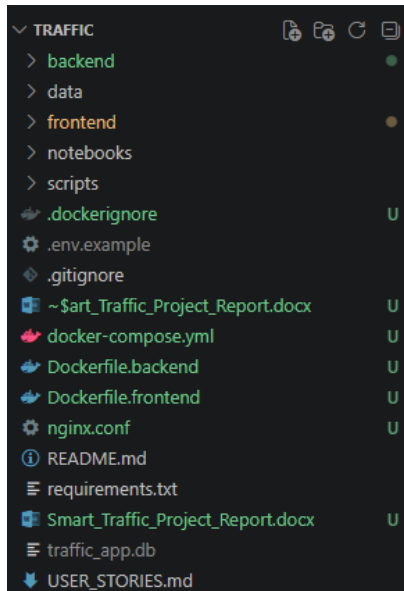
The Smart Traffic Prediction and Route Optimization system is an ML-driven web application for Bengaluru that predicts real-time traffic conditions and recommends the best route between any two locations. Combining a trained XGBoost model on the Bangalore dataset with live weather, geocoding, and road-routing services, the platform provides users with fastest, shortest, and eco-friendly route alternatives displayed on an interactive map.

1.1 User Stories

Actor	User Story
Anushka Ojha	I want to collect and preprocess Bengaluru traffic data, engineer time-based features, and train an XGBoost traffic prediction model so that the system has a reliable ML model.
Mehul Agarwal	I want to evaluate the trained model, implement the shortest-path routing algorithm, build the Route API, and create the map dashboard so that users can view optimal routes.
Mayank Chopra	I want to implement real-time updates, maps integration, data storage, model optimization, and UI enhancements so that the platform is production ready.

2. Technical Architecture

2.1 Project Structure



2.2 Database Tables

Table	Description
traffic_readings	Stores historical Bangalore records: area, road, hour, traffic volume, average speed, congestion level, travel time index.
weather_cache	Caches OpenWeatherMap responses (temperature, precipitation, wind speed, visibility, condition) to reduce external API calls.
route_history	Persists every trip planned by a user: origin, destination, vehicle type, optimize preference, travel time, distance, and CO2 emissions.

2.3 API Endpoints Overview

Method	Endpoint	Description
GET	/api/v1/health	Backend status and model load state
GET	/api/v1/places?query=<text>	Bengaluru place autocomplete (builtin + Nominatim)
GET	/api/v1/weather/{city}	Current weather data with DB caching
GET	/api/v1/predict/{area_name}	Predict traffic volume, speed, congestion for area/time
POST	/api/v1/routes/optimal	Resolve places, predict traffic, compute best route
POST	/api/v1/routes/compare	Returns all 3 route alternatives for side-by-side comparison
POST	/api/v1/routes/recalculate	Re-compute routes mid-trip; returns reroute if > 3 min saved

Case-1: Anushka: I want to collect and preprocess traffic data, engineer meaningful features, and train a traffic prediction model so that the system can deliver accurate, data-driven traffic estimates.

1. Data Collection: US-01

Scenario: The raw Bangalore CSV traffic dataset is ingested into the system. The backend loads this data at startup and seeds it into the SQLite database, making historical traffic records available for fallback predictions and analysis.

Output:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	Date	Area Name	Road/Intersect	Traffic Vol	Average Spd	Travel Time (s)	Congestion Level	Road Capacity Util	Incident Repor	Environmental	Public Transport	Traffic Signal	Parking Usage	Pedestrian and Wheelchair Access
2	01-01-2022	Indiranagar	100 Feet Road	50590	50.2303	1.5	100	100	0	151.18	70.6323302	84.04460047	85.40362889	111 Clear
3	01-01-2022	Indiranagar	CMH Road	30825	29.37712	1.5	100	100	1	111.65	41.9248989	91.40703846	59.98368911	100 Clear
4	01-01-2022	Whitefield	Marathahalli B	7399	54.4744	1.039068846	28.34799386	36.39652494	0	64.798	44.66238371	61.37554084	95.4660201	189 Clear
5	01-01-2022	Koramangala	Sony World Jun	60874	43.81761	1.5	100	100	1	171.748	32.77312315	75.54709159	63.56745159	111 Clear
6	01-01-2022	Koramangala	Sarjapur Road	57292	41.11676	1.5	100	100	3	164.584	35.092601	64.63476238	93.15517129	104 Clear
7	01-01-2022	M.G. Road	Trinity Circle	47848	34.24196	1.5	100	100	3	145.696	39.92787129	61.01676507	55.39457135	94 Over
8	01-01-2022	M.G. Road	Anil Kumble Cir	36574	29.98243	1.5	100	100	3	123.148	72.47912989	81.57368968	90.37200776	115 Clear
9	01-01-2022	Jayanagar	Jayanagar 4th E	25379	38.45518	1.5	79.03882288	100	2	100.758	46.31534352	88.12075836	68.18148012	92 Clear
10	01-01-2022	Jayanagar	South End Circle	25022	35.03937	1.5	78.97959595	100	1	100.044	44.26169322	99.42601816	62.10276358	105 Clear
11	01-01-2022	Hebbal	Hebbal Flyover	31760	56.90456	1.5	97.67246207	100	1	113.52	12.85425991	83.63571773	83.87821809	91 Clear
12	01-01-2022	Hebbal	Ballari Road	38446	26.14488	1.5	100	100	3	126.892	48.86405681	81.18602313	62.09261455	84 Over
13	01-01-2022	Yeshwanthpur	Yeshwanthpur I	15043	47.96985	1.35483457	55.74084425	76.63531646	0	80.086	21.31400999	95.94216754	80.32145298	105 Clear
14	02-01-2022	Indiranagar	100 Feet Road	22050	52.87085	1.130159879	78.4298281	100	3	94.1	35.74010621	70.60809471	62.19948217	115 Fog
15	02-01-2022	Indiranagar	CMH Road	37877	26.42842	1.5	100	100	1	125.754	75.8321009	98.15714308	95.74321951	96 Over
16	02-01-2022	Koramangala	Sarjapur Road	29106	43.56399	1.5	90.40985493	100	3	108.212	53.05050587	99.602154	57.00420076	100 Fog
17	02-01-2022	M.G. Road	Trinity Circle	53217	37.57207	1.5	100	100	5	156.434	33.65966098	75.02331811	54.69909699	102 Clear
18	02-01-2022	M.G. Road	Anil Kumble Circle	37584	45.88951	1.5	100	100	2	125.168	15.97432255	62.06726885	76.56773158	101 Fog
19	02-01-2022	Jayanagar	Jayanagar 4th E	16676	53.52433	1.175588568	47.15579429	78.63350743	2	83.352	38.62670611	66.9317728	57.82185213	92 Fog
20	02-01-2022	Jayanagar	South End Circle	37509	51.51351	1.5	100	100	1	125.018	63.04922773	60.57573955	55.80363203	76 Over
21	02-01-2022	Hebbal	Hebbal Flyover	6758	34.75612	1.069280748	21.20369965	32.77504734	1	63.516	13.17128068	74.98450459	81.29299579	200 Rain
22	02-01-2022	Hebbal	Ballari Road	30209	31.14686	1.5	100	100	2	110.418	48.19317525	97.65859235	69.30513189	90 Clear
23	02-01-2022	Yeshwanthpur	Yeshwanthpur I	20375	40.06999	1.337950124	58.38210538	100	0	90.75	18.27153793	87.86948661	81.44714234	113 Over
24	02-01-2022	Yeshwanthpur	Tumkur Road	15636	32.7943	1.310565184	48.93607081	82.48757326	1	81.272	62.56097475	90.18171496	55.15619344	115 Over
Bangalore_traffic_Dataset														
Ready Accessibility: Unavailable														

2. Data Preprocessing: US-02

Scenario: The raw data is cleaned, normalised, and transformed into a structured format ready for feature engineering and model training. Null rows, duplicate entries, and inconsistent column names are handled.

Output:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
1	date	area_name	road_name	traffic_vol	average_s	travel_time	congestion	hour	day_of_w	month	is_weekend	weather_c	traffic_vol	average_s	traffic_vol	average_speed	rolling_3			
2	01-01-2022	Indiranag	100 Feet R	50590	50.2303	1.5	100	0	5	1	1	Clear	50590	50.2303	50590	50.23				
3	02-01-2022	Indiranag	100 Feet R	22050	52.87085	1.13016	78.42983	0	6	1	1	Fog	50590	50.2303	50590	50.23				
4	03-01-2022	Indiranag	100 Feet R	23457	38.86905	1.380025	87.20214	0	0	1	0	Clear	22050	52.87085	36320	51.55				
5	04-01-2022	Indiranag	100 Feet R	46199	41.2285	1.5	100	0	1	1	0	Clear	23457	38.86905	32032.33	47.32				
6	05-01-2022	Indiranag	100 Feet R	49952	50.80134	1.5	100	0	2	1	0	Overcast	46199	41.2285	30568.67	44.32				
7	06-01-2022	Indiranag	100 Feet R	46556	46.5933	1.5	100	0	3	1	0	Clear	49952	50.80134	39869.33	43.63				
8	07-01-2022	Indiranag	100 Feet R	24405	29.84256	1.237176	82.95172	0	4	1	0	Clear	46556	46.5933	47569	46.21				
9	08-01-2022	Indiranag	100 Feet R	36384	28.18948	1.5	100	0	5	1	1	Rain	24405	29.84256	40304.33	42.41				
10	10-01-2022	Indiranag	100 Feet R	43024	28.07095	1.5	100	0	0	1	0	Overcast	36384	28.18948	35781.67	34.88				
11	11-01-2022	Indiranag	100 Feet R	11423	20	1.007677	33.27293	0	1	1	0	Clear	43024	28.07095	34604.33	28.7				
12	13-01-2022	Indiranag	100 Feet R	11371	51.02477	1.103448	31.19746	0	3	1	0	Clear	11423	20	30277	25.42				
13	14-01-2022	Indiranag	100 Feet R	24545	51.50382	1.224176	74.03646	0	4	1	0	Overcast	11371	51.02477	21939.33	33.03				
14	15-01-2022	Indiranag	100 Feet R	20081	52.32649	1.291669	69.9825	0	5	1	1	Fog	24545	51.50382	15779.67	40.84				
15	16-01-2022	Indiranag	100 Feet R	37666	35.89581	1.5	100	0	6	1	1	Clear	20081	52.32649	18665.67	51.62				
16	17-01-2022	Indiranag	100 Feet R	43778	45.58485	1.5	100	0	0	1	0	Clear	37666	35.89581	27430.67	46.58				
17	19-01-2022	Indiranag	100 Feet R	37647	61.34222	1.5	100	0	2	1	0	Clear	43778	45.58485	33841.67	44.6				
18	20-01-2022	Indiranag	100 Feet R	34920	21.73952	1.5	100	0	3	1	0	Windy	37647	61.34222	39697	47.61				
19	21-01-2022	Indiranag	100 Feet R	37614	36.84248	1.5	100	0	4	1	0	Overcast	34920	21.73952	38781.67	42.89				
20	22-01-2022	Indiranag	100 Feet R	38113	42.89057	1.5	100	0	5	1	1	Fog	37614	36.84248	36727	39.97				
21	23-01-2022	Indiranag	100 Feet R	32690	31.87457	1.5	100	0	6	1	1	Overcast	38113	42.89057	36862.33	33.82				
22	24-01-2022	Indiranag	100 Feet R	12829	44.72668	1.016024	38.59255	0	0	1	0	Clear	32690	31.87457	36139	37.2				

3. Feature Engineering: US-03

Scenario: Temporal and contextual features are derived from the cleaned dataset so that the ML model can learn patterns tied to time of day, day of week, and real-world conditions directly improving prediction accuracy.

Output:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
1	date	area_name	road_name	traffic_vol	average_speed	travel_time	congestion	hour	day_of_week	month	is_weekend	weather	traffic_vol	average_speed	traffic_vol	average_speed	rolling_3			
2	01-01-2022	Indiranagar	100 Feet R	50590	50.2303	1.5	100	0	5	1	1	Clear	50590	50.2303	50590	50.23				
3	02-01-2022	Indiranagar	100 Feet R	22050	52.87085	1.13016	78.42983	0	6	1	1	Fog	50590	50.2303	50590	50.23				
4	03-01-2022	Indiranagar	100 Feet R	23457	38.86905	1.380025	87.20214	0	0	1	0	Clear	22050	52.87085	36320	51.55				
5	04-01-2022	Indiranagar	100 Feet R	46199	41.2285	1.5	100	0	1	1	0	Clear	23457	38.86905	32032.33	47.32				
6	05-01-2022	Indiranagar	100 Feet R	49952	50.80134	1.5	100	0	2	1	0	Overcast	46199	41.2285	30568.67	44.32				
7	06-01-2022	Indiranagar	100 Feet R	46556	46.5933	1.5	100	0	3	1	0	Clear	49952	50.80134	39869.33	43.63				
8	07-01-2022	Indiranagar	100 Feet R	24405	29.84256	1.237176	82.95172	0	4	1	0	Clear	46556	46.5933	47569	46.21				
9	08-01-2022	Indiranagar	100 Feet R	36384	28.18948	1.5	100	0	5	1	1	Rain	24405	29.84256	40304.33	42.41				
10	10-01-2022	Indiranagar	100 Feet R	43024	28.07095	1.5	100	0	0	1	0	Overcast	36384	28.18948	35781.67	34.88				
11	11-01-2022	Indiranagar	100 Feet R	11423	20	1.007677	33.27293	0	1	1	0	Clear	43024	28.07095	34604.33	28.7				
12	13-01-2022	Indiranagar	100 Feet R	11371	51.02477	1.103448	31.19746	0	3	1	0	Clear	11423	20	30277	25.42				
13	14-01-2022	Indiranagar	100 Feet R	24545	51.50382	1.224176	74.03646	0	4	1	0	Overcast	11371	51.02477	21939.33	33.03				
14	15-01-2022	Indiranagar	100 Feet R	20081	52.32649	1.291669	69.9825	0	5	1	1	Fog	24545	51.50382	15779.67	40.84				
15	16-01-2022	Indiranagar	100 Feet R	37666	35.89581	1.5	100	0	6	1	1	Clear	20081	52.32649	18665.67	51.62				
16	17-01-2022	Indiranagar	100 Feet R	43778	45.58485	1.5	100	0	0	1	0	Clear	37666	35.89581	27430.67	46.58				
17	19-01-2022	Indiranagar	100 Feet R	37647	61.34222	1.5	100	0	2	1	0	Clear	43778	45.58485	33841.67	44.6				
18	20-01-2022	Indiranagar	100 Feet R	34920	21.73952	1.5	100	0	3	1	0	Windy	37647	61.34222	39697	47.61				
19	21-01-2022	Indiranagar	100 Feet R	37614	36.84248	1.5	100	0	4	1	0	Overcast	34920	21.73952	38781.67	42.89				
20	22-01-2022	Indiranagar	100 Feet R	38113	42.89057	1.5	100	0	5	1	1	Fog	37614	36.84248	36727	39.97				
21	23-01-2022	Indiranagar	100 Feet R	32690	31.87457	1.5	100	0	6	1	1	Overcast	38113	42.89057	36882.33	33.82				
22	24-01-2022	Indiranagar	100 Feet R	12829	44.72668	1.016024	38.59255	0	0	1	0	Clear	32690	31.87457	36139	37.2				

4. Traffic Prediction Model: US-04

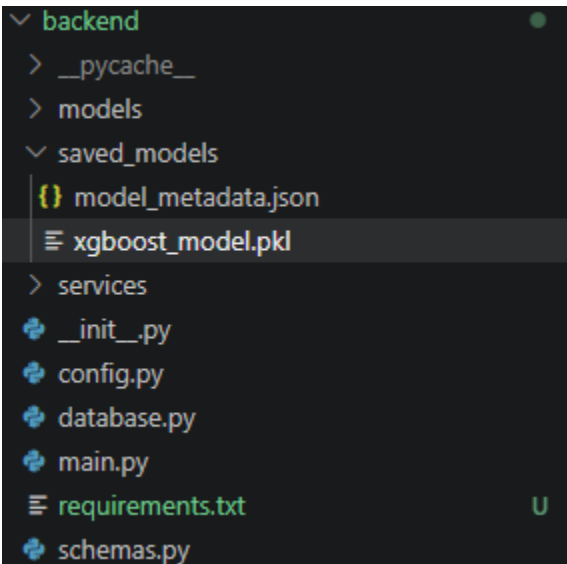
Scenario: A trained XGBoost model predicts traffic volume for any Bengaluru area, returning congestion level and average speed alongside the raw volume estimate. The model is trained in Google Colab and exported as a pickle artifact loaded by the FastAPI backend at startup.

Output:

```

backend > models > predictor.py > ...
22
23
24 class TrafficPredictor:
25     def __init__(self) -> None:
26         self.model_path = MODEL_DIR / "xgboost_model.pkl"
27         self.meta_path = MODEL_DIR / "model_metadata.json"
28         self.load_error: str | None = None
29         self.area_stats, self.area_templates = self._load_area_stats()
30         self.model = self._load_pickle(self.model_path)
31         self.metadata = self._load_metadata()
32
33     def predict_for_area(
34         self,
35         area_name: str,
36         hour: int,
37         day: int,
38         weather: dict[str, Any] | None = None,
39         vehicle_type: str = "car",
40         requested_location: str | None = None,
41     ) -> dict[str, Any]:
42         weather = weather or {}
43         baseline = self.area_stats.get(area_name) or self.area_stats.get("city_average") or {
44             "traffic_volume": 20000.0,
45             "average_speed": 32.0,
46             "congestion_level": 55.0,
47         }
48
49         features = {
50             "hour": hour,
51             "day_of_week": day,
52             "is_weekend": 1 if day >= 5 else 0,
53             "temperature": float(weather.get("temperature", 27.0)),
54             "precipitation": float(weather.get("precipitation", 0.0)),
55             "wind_speed": float(weather.get("wind_speed", 8.0)),

```



Objective

Establish the data and ML layer from raw CSV to a live trained predictor so that all routing and dashboard features are powered by real traffic data.

Key Analysis

- Implement a complete data ingestion pipeline.
- Build a preprocessing script that cleans, normalises, and outputs a model-ready CSV.
- Engineer 97 features including temporal, lag, rolling, and categorical encodings from the raw traffic data.
- Train an XGBoost pipeline in Google Colab and export it as a pickle artifact consumed by FastAPI.

Outcomes

- Trained XGBoost model predicting traffic volume, speed, and congestion for Bengaluru.
- Model source flag in API response clearly indicates whether trained artifact or heuristic fallback is active.

Case-2: Mehul: I want to evaluate the prediction model, implement the shortest-path routing algorithm, build the Route API, and create the map dashboard so that users can see optimal routes with live traffic on an interactive map.

1. Model Evaluation: US-05

Scenario: The trained XGBoost model is evaluated on the test split to validate that prediction quality meets the acceptance criteria before the model is deployed to the backend.

Output:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model.fit(X_train, y_train)
predictions = model.predict(X_test)

mae = mean_absolute_error(y_test, predictions)
rmse = mean_squared_error(y_test, predictions, squared=False)
mape = ((y_test - predictions).abs() / y_test.clip(lower=1)).mean() * 100
print({'MAE': mae, 'RMSE': rmse, 'MAPE': mape})
```

Metric	Value	Interpretation
MAE	4,309.95	Mean absolute error in traffic volume units
RMSE	6,758.82	Root mean squared error
MAPE	13.03 %	Model is off by ~13 % on average

2. Shortest Path Algorithm: US-06

Scenario: Given an origin and destination in Bengaluru, the system computes multiple real-road route alternatives and selects the optimal one based on the user's chosen optimisation goal: fastest time, shortest distance, or lowest CO2.

- Each route's raw travel time is adjusted by three multiplicative factors:
- Traffic factor: derived from the XGBoost prediction for the resolved area

- Output:**

The screenshot displays the Traffic Optimizer web application interface. The left sidebar contains the following elements:

- Traffic Optimizer** header with the Bengaluru Route Intelligence logo.
- API status: API connected (green dot) and Model ready (green dot).
- Light mode toggle (orange dot).
- TRIP PLANNER section with:
 - Origin: MG Road (12,9756, 77.6050)
 - Destination: Electronic City (12.8399, 77.6770)
 - Optimize for: Fastest (dropdown menu)
 - Vehicle: Car (dropdown menu with options: Car, Bike, Truck)
 - Find best route button (green)

The main map area shows Bengaluru with a red route highlighted from MG Road to Electronic City. The map includes various landmarks, roads, and a scale bar at the bottom right.

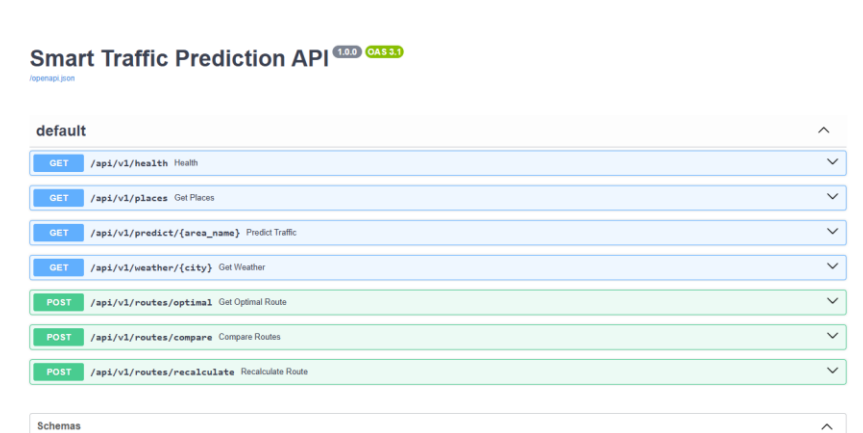
3. Route API: US-07

Scenario: Frontend clients call REST endpoints to get route recommendations, compare alternatives, and trigger mid-trip recalculation all through a single versioned FastAPI service.

Endpoint	Request Body	Response Fields
POST /api/v1/routes/optimal	origin, destination, vehicle_type, optimize_for	routes[], traffic_prediction, weather, best_route_index, resolved coords
POST /api/v1/routes/compare	Same as /optimal	All 3 route alternatives with full time, distance, CO2, congestion metrics
POST /api/v1/routes/recalculate	origin, destination, vehicle_type, current_route_index	new_routes[], reroute_available (bool), reroute_message, time_saved_minutes

Output:

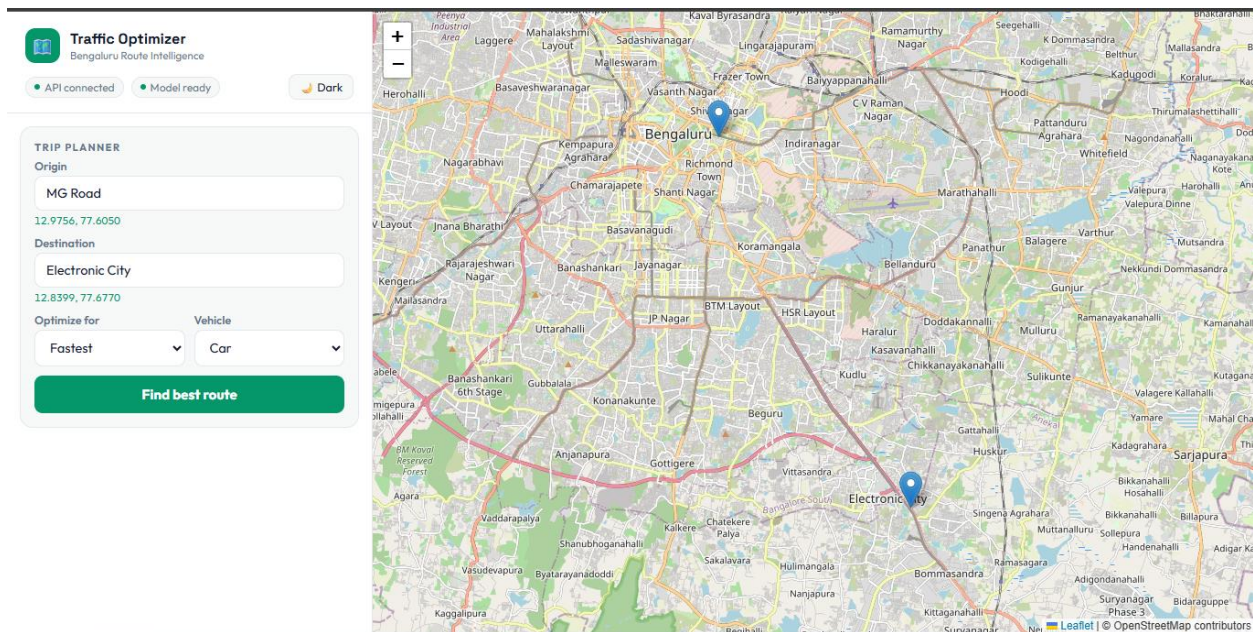
```
Pretty-print ✓  
  
{  
  "status": "ok",  
  "timestamp": "2026-04-29T09:45:29.841174",  
  "artifact_loaded": true,  
  "trained_model_active": true,  
  "artifact_path": "D:\\Traffic\\backend\\saved_models\\xgboost_model.pkl",  
  "load_error": null  
}
```



4. Map Dashboard: US-08

Scenario: After entering origin and destination, the user sees a split dashboard: a search panel on the left and an interactive map on the right, with route comparison cards and a live traffic prediction panel so that routes and traffic are shown correctly.

Output:



Objective

Take the trained model to production by evaluating it, building the routing algorithm, exposing everything through a clean API, and delivering the interactive map dashboard users will actually see and use.

Key Analysis

- Evaluate the XGBoost model with MAE, RMSE, and MAPE on a proper train-test split.
- Implement a routing service that combines OSRM road routes with traffic, weather, and vehicle penalties.
- Build three versioned FastAPI endpoints (optimal, compare, recalculate) with full Pydantic validation.

Outcomes

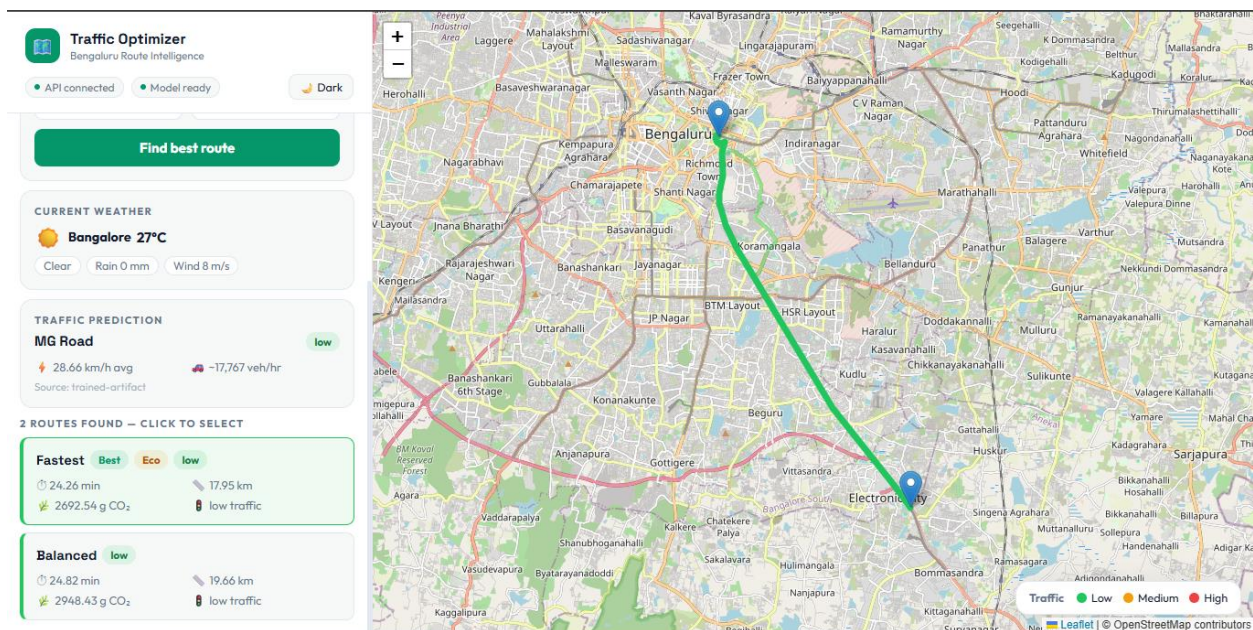
- Model evaluation metrics (MAE=4309.95, RMSE=6758.82, MAPE=13.03%) confirm the predictor is fit for deployment.
- Route alternatives adjusted by traffic prediction, weather, and vehicle type returned per API request.
- Interactive map dashboard rendering three colour-coded route options with live traffic and weather data.

Case-3: Mayank: As a full-stack developer, I want to implement real-time route updates, maps integration, data storage, model optimisation, and UI enhancements so that the platform is polished, responsive, and production-ready.

1. Real-Time Updates: US-09

Scenario: While viewing their active route, users are automatically notified when traffic conditions change enough to offer a faster path. The system polls the backend every 30 seconds and reflects updated traffic data dynamically.

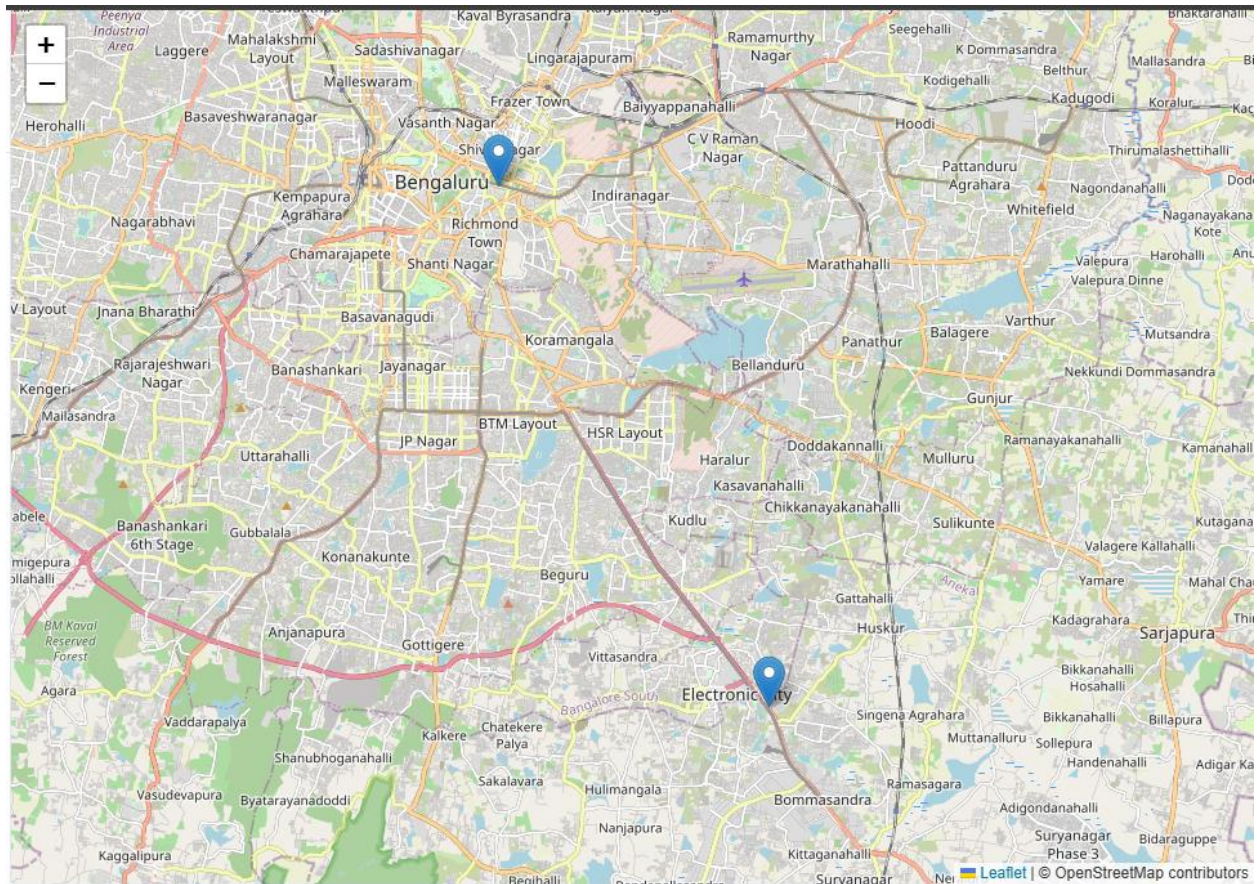
Output:



2. Google Maps Integration: US-10

Scenario: Routes are drawn on a live map tile layer so that users can visually verify their journey path. Origin and destination markers are clearly visible and the map loads reliably for all Bengaluru locations.

Output:



3. Data Storage: US-11

Scenario: All traffic readings, weather lookups, and user trip history are persisted in a structured database so that the system has reliable fallback data.

- **weather_cache:** OpenWeatherMap responses with timestamp, city, and all-weather fields — queried before hitting the live API to reduce latency
- **route_history:** every trip planned (origin, destination, vehicle_type, optimize_for, travel_time, distance, co2_emissions, created_at)

Output:

```

SCHEMA_STATEMENTS = [
    """
    CREATE TABLE IF NOT EXISTS traffic_readings (
        id INTEGER PRIMARY KEY,
        recorded_at TEXT,
        hour INTEGER,
        area_name TEXT,
        road_name TEXT,
        traffic_volume REAL,
        average_speed REAL,
        congestion_level TEXT,
        travel_time_index REAL,
        source TEXT DEFAULT 'bangalore'
    )
    """,
    """
    CREATE TABLE IF NOT EXISTS weather_cache (
        id INTEGER PRIMARY KEY,
        city TEXT,
        timestamp TEXT,
        temperature REAL,
        precipitation REAL,
        wind_speed REAL,
        visibility REAL,
        condition TEXT
    )
    """,
    """
    CREATE TABLE IF NOT EXISTS route_history (
        id INTEGER PRIMARY KEY,
        origin TEXT,
        destination TEXT,
        vehicle_type TEXT,
        optimize_for TEXT,
        """

```

4. Model Optimization: US-12

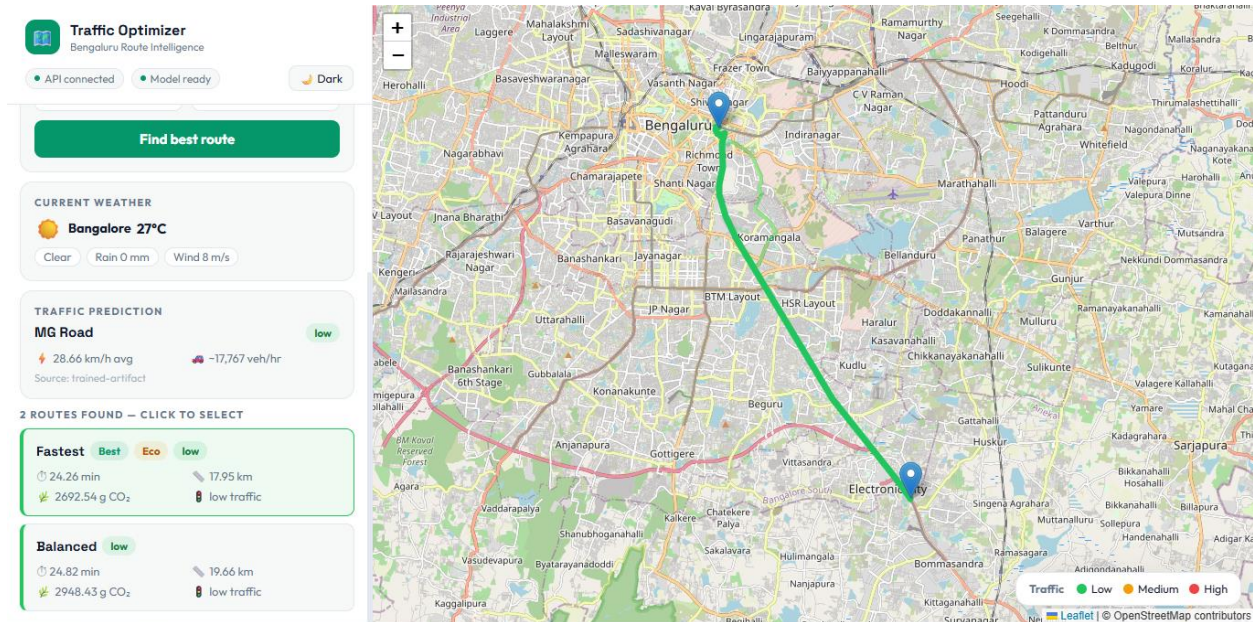
Scenario: The XGBoost model's hyperparameters are tuned so that prediction accuracy is maximised beyond the baseline. The optimised artifact replaces the initial model in backend/saved_models/ without any code changes.

Output: The training section supports grid search over key XGBoost parameters: n_estimators, max_depth, learning_rate, subsample, and colsample_bytree.

5. UI Enhancements: US-13

Scenario: The dashboard is fully responsive and accessible in both light and dark themes, with place autocomplete, clear loading states, error banners, and filter controls so that users have a smooth and intuitive experience.

Output:



6. Deployment: US-14

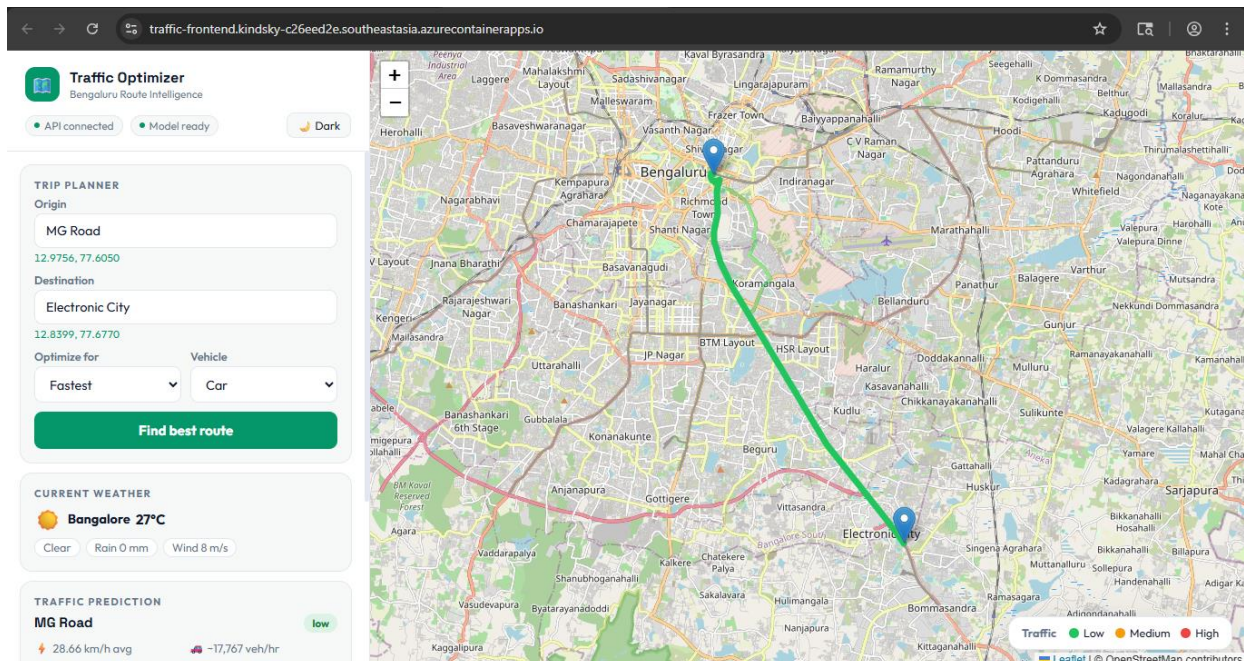
Scenario: Deployed the Frontend and Backend on Azure using docker

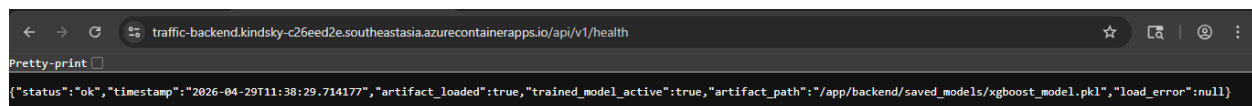
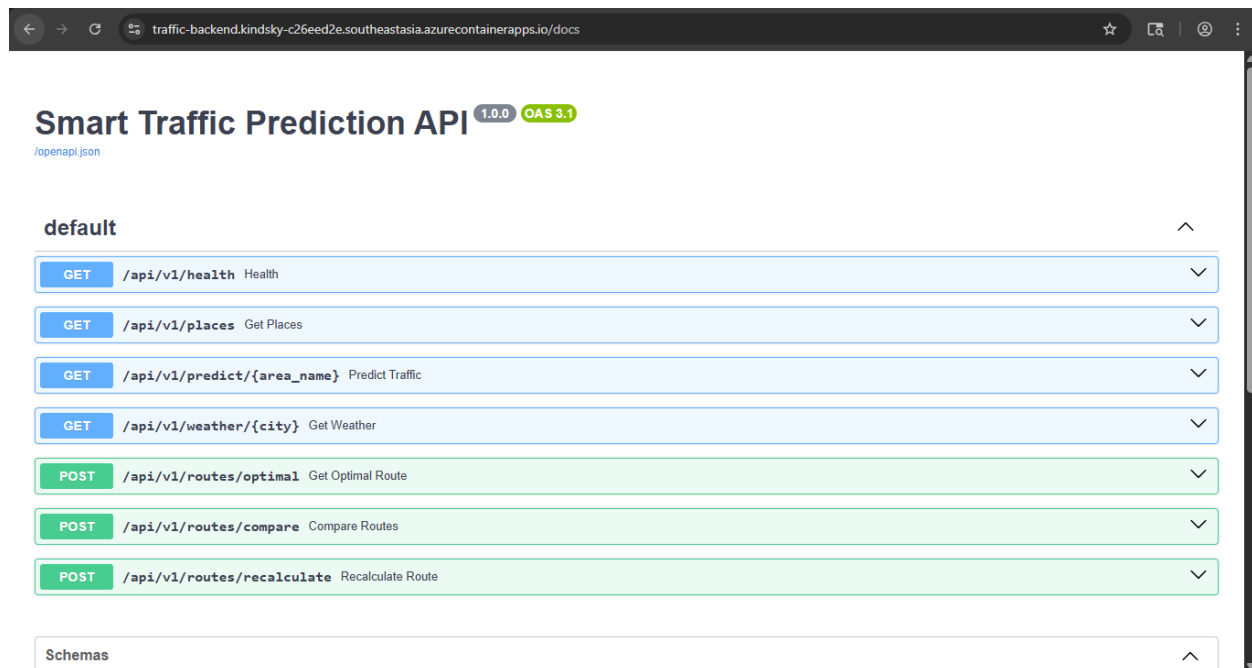
Frontend link: <https://traffic-frontend.kinsky-c26eed2e.southeastasia.azurecontainerapps.io/>

Backend link: <https://traffic-backend.kinsky-c26eed2e.southeastasia.azurecontainerapps.io/docs>

Backend health link : <https://traffic-backend.kinsky-c26eed2e.southeastasia.azurecontainerapps.io/api/v1/health>

Output:





Objective

Deliver real-time route intelligence, a reliable persistence layer, an optimised ML model, and a polished responsive UI so that the platform is complete and production-ready.

Key Analysis

- Integrate react-leaflet with OpenStreetMap for congestion-coloured polylines, markers, and auto-fit bounds.
- Structure the Colab notebook for XGBoost hyperparameter tuning with grid search over key parameters.
- Build a responsive dark/light UI with place autocomplete, loading states, error banners, and health status.

Outcomes

- Live reroute suggestions appear automatically when a faster path saves more than 3 minutes.
- Interactive map with colour-coded polylines, custom markers, and legend renders all Bengaluru routes correctly.
- All trips, weather lookups, and traffic readings persisted in SQLite and available for fallback and analytics.

- Tunable training pipeline supports repeated optimisation without any changes to backend inference code.
- Polished dark/light UI with place autocomplete and health status indicator delivers a smooth user experience.

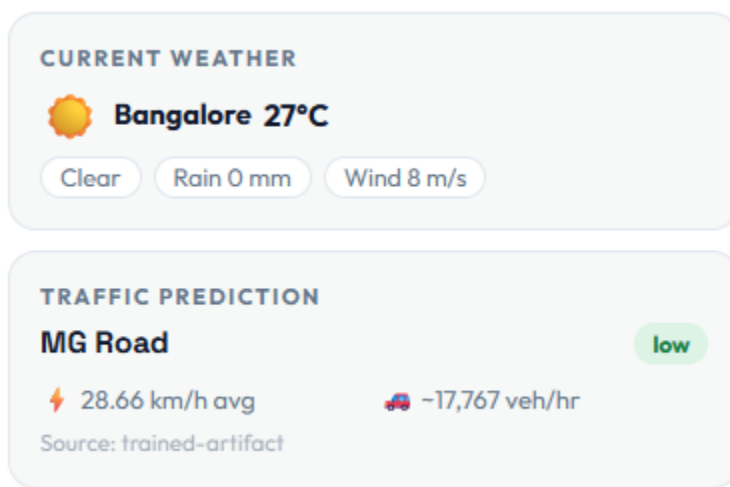
Additional Features

The following features were implemented collaboratively by the full team as value-added enhancements beyond the core user stories.

1. Weather-Aware Routing: US-15

Scenario: Current weather conditions are fetched for the trip location and factored into travel time calculations so that rain, wind, or low visibility automatically increases the estimated journey time before routes are returned.

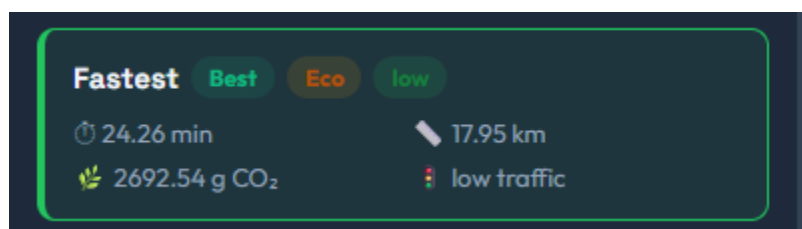
Output:



2. Eco-Friendly Routes: US-16

Scenario: Each route alternative displays its estimated CO2 output in grams so that environmentally conscious users can choose the greenest path, and selecting "eco" as the optimise goal promotes that route as the recommended choice.

Output:



3. Vehicle Type Selection: US-17

Scenario: Users select their vehicle type (car, bike, or truck) before searching, and the system adjusts both traffic predictions and route travel times according to vehicle-specific speed and emission characteristics.

Output:

2 ROUTES FOUND — CLICK TO SELECT

Fastest **Best** **Eco** **low**

🕒 24.26 min 📏 17.95 km

🌿 2692.54 g CO₂ 🚦 low traffic

Balanced **low**

🕒 24.82 min 📏 19.66 km

🌿 2948.43 g CO₂ 🚦 low traffic

4. Alternative Route Trade-offs: US-18

Scenario: Users see up to three distinct route alternatives side-by-side so they can consciously trade off journey time, distance, and emissions before committing to a path.

Output:

2 ROUTES FOUND — CLICK TO SELECT

Fastest **Best** **Eco** **low**

🕒 24.26 min 📏 17.95 km

🌿 2692.54 g CO₂ 🚦 low traffic

Balanced **low**

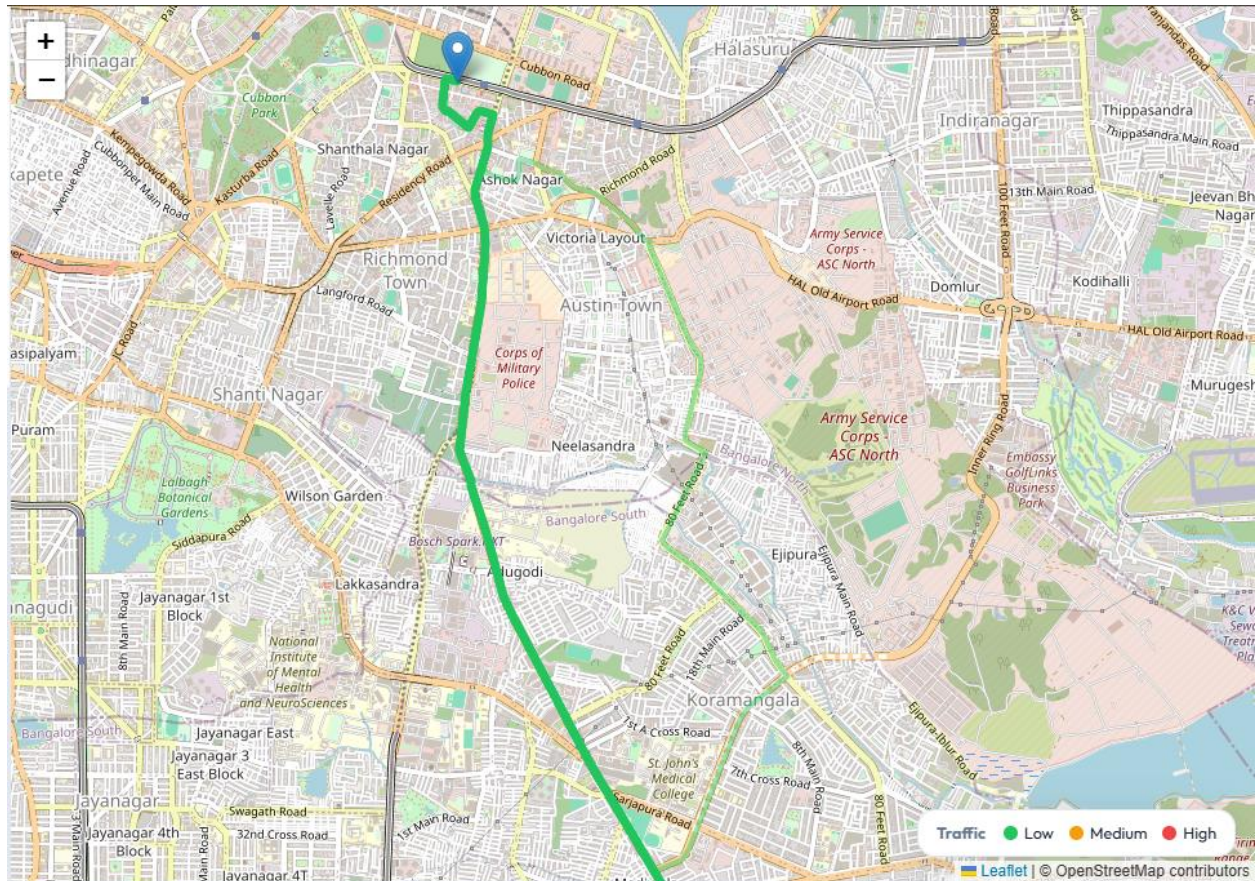
🕒 24.82 min 📏 19.66 km

🌿 2948.43 g CO₂ 🚦 low traffic

5. Mid-Trip Optimal Rerouting: US-19

Scenario: While navigating, the system recommends a better route when traffic conditions change enough to save meaningful time, allowing users to switch with a single tap without re-entering their trip details.

Output:



Objective

Extend the core platform with weather intelligence, eco scoring, vehicle personalisation, multi-route comparison, and live rerouting so that users get a complete, real-world navigation experience.

Key Analysis

- Integrate live weather data with tiered penalties that adjust all route ETAs automatically.
- Allow vehicle type to influence predictions, route timing, and emissions simultaneously.
- Implement a polling-based reroute flow that is non-disruptive (toast alert) and triggers only when savings are meaningful.

Outcomes

- Weather-adjusted ETAs that reflect rain, wind, and visibility conditions automatically.
- CO2 scores on every route with Eco badge and eco-optimised routing mode.
- Three comparable route cards with time, distance, CO2, and congestion metrics displayed simultaneously.